Like all matchers, compound matchers can be passed as arguments to other matchers:

10-exploring-rspec-expectations/04/composing_matchers.rb

```
letter_ranges = ['N to Z', 'A to M']
expect(letter_ranges).to contain_exactly(
  a_string_starting_with('A') & ending_with('M'),
  a_string_starting_with('N') & ending_with('Z')
)
```

You can mix and match these techniques for composing matchers, all in one expectation. You can also nest them as deeply as you need. Here, we're combining matchers with logical operators, then nesting those combinations into a collection matcher. This flexibility allows you to describe complex data precisely.

Now that you've seen how to combine matchers, let's turn our attention to output.

Generated Example Descriptions

Matchers have another useful ability over simpler assert methods: they're self-describing. The matcher protocol includes the optional (but recommended) description method. All built-in matchers define this method:

```
>> start_with(1).description
=> "start with 1"
>> (start_with(1) & end_with(9)).description
=> "start with 1 and end with 9"
>> contain_exactly( a_string_starting_with(1) & ending_with(9) ).description
=> "contain exactly (a string starting with 1 and ending with 9)"
```

As you can see, the descriptions of composed and compound matchers include the description of each part. These descriptions are used in failure messages when you pass one matcher to another. They can also help you reduce duplication in your specs. Typically, each example has a string stating the intended behavior:

10-exploring-rspec-expectations/06/spec/cookie_recipe_spec.rb

```
class CookieRecipe
  attr_reader :ingredients

  def initialize
    @ingredients = [:butter, :milk, :flour, :sugar, :eggs, :chocolate_chips]
  end
end

RSpec.describe CookieRecipe, '#ingredients' do
  it 'should include :butter, :milk and :eggs' do
    expect(CookieRecipe.new.ingredients).to include(:butter, :milk, :eggs)
  end
```